

StackNest - Community Discussion Platform

Documentation

Introduction

StackNest is a full-stack community discussion platform inspired by modern social discussion websites like Reddit and Stack Overflow.

The platform allows users to:

1. Create communities
2. Publish discussion posts
3. Upload images with posts
4. Vote on posts (upvote/ downvote)
5. Comment on discussions
6. Comment on discussions
7. Browse community-specific content
8. Authenticate securely using JWT

StackNest focuses on creating organized discussion spaces where users can share knowledge, ask questions, and engage with communities around specific interests.

Design Inspiration

This project draws inspiration from modern discussion platforms:

1. Reddit
 - Community-based discussions
 - Voting system

- User-generated content

2. Stack Overflow

- Knowledge sharing
- Technical discussions
- Community-driven engagement

3. StackNest Custom Implementation

- JWT Authentication
- Community Management
- Cloudinary Image Uploads
- PostgreSQL Relational Database
- Vote Aggregation System

Roles

User :

- Register account
- Login securely
- Create communities
- Create posts
- Upload images
- Upvote/downvote posts
- Comment on discussions
- Browse communities

Use Cases of the Project

Key Use Cases

User Registration

1. User creates account
2. Password is encrypted and stored securely

User Authentication

1. User logs in
2. JWT token is generated
3. Protected APIs are accessed using JWT

Community Management

1. Create communities
2. View all communities
3. Access communities through unique slugs

Post Management

1. Create posts
2. Upload images
3. Browse discussion feeds
4. View post details

Voting System

1. Upvote posts
2. Downvote posts
3. Remove votes
4. Change existing votes

Comment System

1. Add comments
2. Participate in discussions
3. View all comments for a post

Tech Stack

1. Backend

- **SpringBoot**

1. Rapid REST API development
2. Dependency Injection
3. Layered Architecture
4. Backend business logic

- **Spring Security**

1. API Protection
2. Authentication Handling
3. Secure endpoint access

- **JWT (Json Web Token)**

1. Stateless authentication
2. Scalable and secure
3. Protected API access

- **PostgreSQL**

1. Relational database
2. Stores users, communities, post, comments, votes.

- **Cloudinary**

1. Image Uploads
2. CDN Storage
3. Image hosting and Delivery

2. Frontend

- **React**

1. Component-based UI
2. Efficient state management
3. Scalable frontend architecture

- **TailWind CSS**

1. Utility-first styling
2. Clean and modern UI
3. Faster development workflow

Authentication

Authentication is implemented using JWT(Json Web Token).

Flow:

1. User send login request :
POST /auth/login
2. Backend validates credentials
3. JWT token is generated
Authorization: Bearer <JWT_TOKEN>
4. Token is used for all protected APIs

Security Layers:

1. JwtFilter : Validates JWT for protected routes
2. ApiKeyFilter : Validates API Key

Flowcharts

◆ **Backend Flow**

- Client
 - JWT Authentication
 - Community Creation / Selection
 - Post Creation
 - Image Upload (Cloudinary)
 - Vote / Comment Interaction
 - Business Logic Processing
 - PostgreSQL Storage
 - Response Returned

◆ **Authentication Flow**

- Client
 - Signup / Login Request
 - Credential Validation
 - JWT Token Generation
 - Token Returned
 - Authorization Header
 - Protected API Access

◆ **Post Creation Flow**

- Client
 - Upload Image
 - Cloudinary Upload
 - Image URL Generated
 - Create Post Request
 - Community Validation
 - Post Stored in Database
 - Response Returned

◆ Voting Flow

- Client
 - Vote Request
 - JWT Validation
 - User Validation
 - Existing Vote Check
 - Add / Update / Remove Vote
 - Vote Count Recalculated
 - Response Returned

◆ Comment Flow

- Client
 - Comment Request
 - JWT Validation
 - Post Validation
 - Comment Creation
 - Database Storage
 - Response Returned

◆ System Architecture

- Controller Layer → Handles API requests
- Service Layer → Business logic and validations
- Security Layer → JWT Authentication and endpoint protection
- Repository Layer → Database operations
- Infrastructure → PostgreSQL + Cloudinary CDN

◆ Database Design (ER Concept)

- User → creates many Communities
- User → creates many Posts

- User → creates many Comments
- User → creates many Votes
- Community → contains many Posts
- Post → belongs to Community
- Post → belongs to User
- Post → has many Comments
- Post → has many Votes
- Comment → belongs to User
- Comment → belongs to Post
- Vote → belongs to User
- Vote → belongs to Post

◆ **Media Upload Architecture**

Client

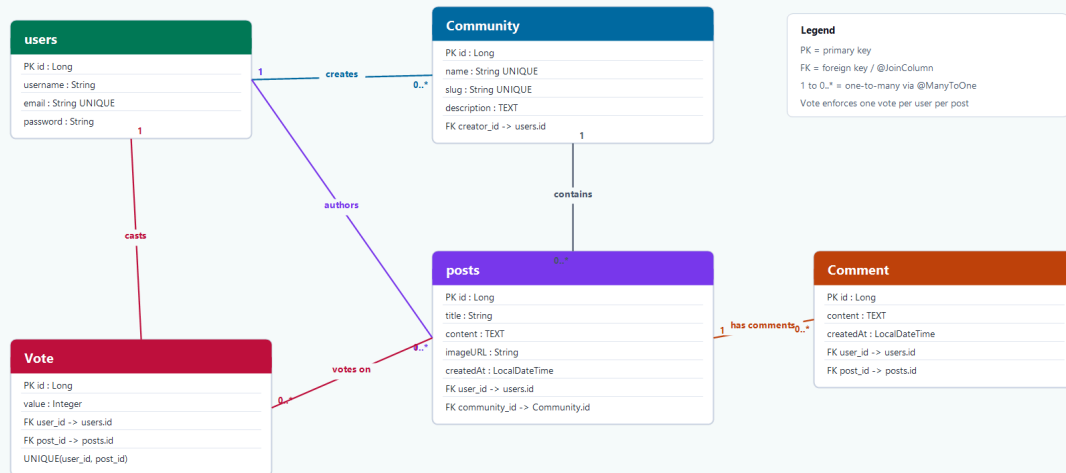
- Multipart Image Upload
- Spring Boot Upload API
- Cloudinary CDN
- Secure Image URL Generated
- URL Stored in Post
- Frontend Renders Image

◆ **Frontend Integration Flow**

- Next.js Frontend
 - REST API Calls
 - Spring Boot Backend
 - PostgreSQL Database
 - Cloudinary CDN
 - JSON Response
 - UI Rendering

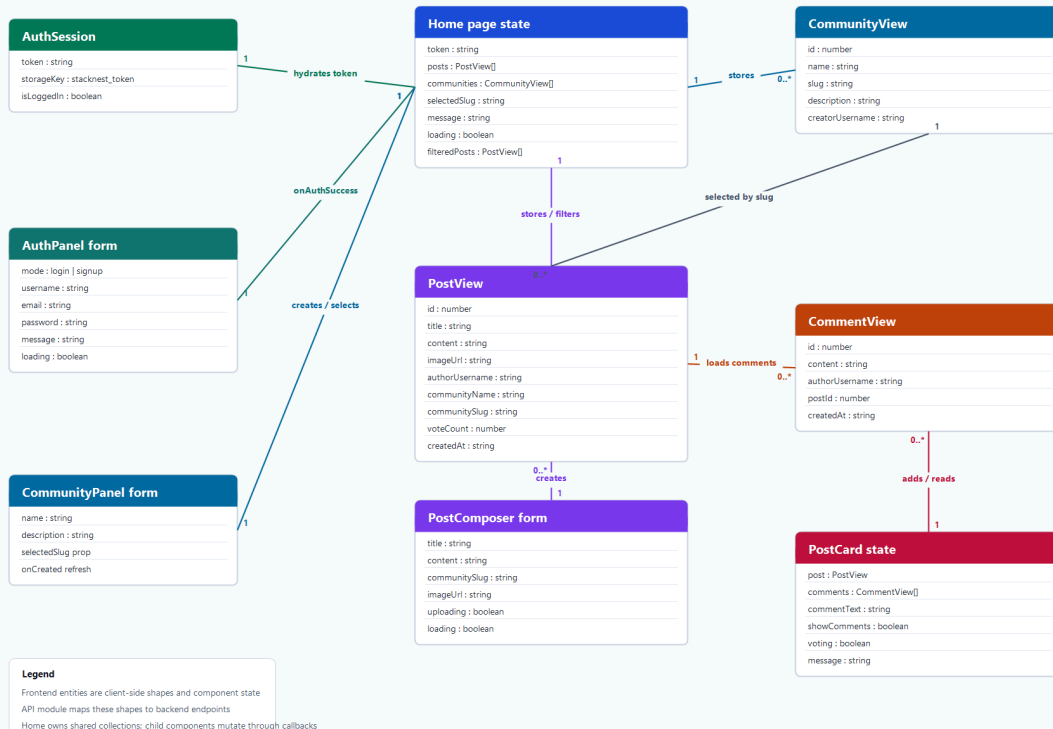
StackNest Backend ER Diagram

Spring Boot JPA entities and database relationships



StackNest Frontend ER Diagram

Next.js client-side data, state, components, and API DTO relationships



Industry Value of the Project

This project demonstrates real-world full-stack software engineering concepts.

Industry Applications

- Community Discussion Platforms (Reddit, Quora)
- Developer Communities (Stack Overflow, Dev.to)
- Knowledge Sharing Platforms
- Social Networking Platforms
- Educational Discussion Forums
- Enterprise Collaboration Platforms

Key Concepts Implemented

- JWT Authentication & Authorization
- Community-Based Content Organization
- RESTful API Development
- Relational Database Design
- DTO-Based Architecture
- Image Upload & Cloud Storage Integration
- Voting & Engagement System
- Discussion & Comment System
- Layered Architecture (Controller → Service → Repository)
- Full-Stack Application Development

Real-world Inspiration Applied

Reddit

- Community-driven discussions
- Upvote/Downvote system
- User-generated content

Stack Overflow

- Knowledge sharing
- Technical discussions
- Community participation

StackNest combines both ideas into a single system:

- Community-focused discussions
- Social engagement through voting
- User-generated content creation
- Media-supported posts
- Secure authentication system

Technologies Used

Backend Technologies

1. Spring Boot

- Used to build RESTful APIs quickly and efficiently
- Provides auto-configuration and embedded server support
- Enables clean layered architecture (Controller → Service → Repository)
- Simplifies dependency management and project setup
- Handles business logic and request processing

2. Spring Security

- Handles authentication and authorization
- Protects API endpoints
- Integrates with JWT authentication
- Provides security filters and endpoint protection
- Ensures secure access to protected resources

3. JWT (JSON Web Token)

- Used for stateless authentication
- Eliminates need for server-side session storage
- Ensures secure communication between client and server
- Passed in Authorization header for protected APIs
- Enables scalable authentication architecture

4. PostgreSQL

- Relational database used for persistent storage
- Stores:
 - Users
 - Communities
 - Posts
 - Comments
 - Votes
- Provides strong data consistency and relational integrity
- Supports complex relationships between entities
- Suitable for scalable social platforms

5. Cloudinary

- Cloud-based image hosting platform
- Stores uploaded post images
- Generates secure CDN URLs
- Provides optimized image delivery
- Eliminates need for local file storage

6. Java 21

- Core programming language for backend development
- Supports modern language features
- Provides improved performance
- Strong ecosystem and Spring Boot compatibility
- Suitable for enterprise-grade applications

Frontend Technologies

1. Next.js

- Used to build modern web applications
- Supports server-side rendering
- Provides routing and optimization features
- Improves application performance
- Enables scalable frontend architecture

2. React

1. Component-based frontend framework
2. Promotes reusable UI components
3. Handles:

- Feed rendering
 - Community pages
 - Post interactions
 - API integration
 - State management
4. Enables dynamic and responsive user interfaces

3. Tailwind CSS

- Utility-first CSS framework
- Allows rapid UI development
- Helps create:
 1. Clean layouts
 2. Responsive design
 3. Modern user interfaces
 4. Mobile-friendly experiences
 5. Reduces need for custom CSS

Integration Overview

Next.js Frontend

↓ (HTTP Requests)

Spring Boot Backend

↓

PostgreSQL (Persistent Data)

↓

Clouinary (Image Storage & CDN)

Conclusion

StackNest successfully demonstrates the design and implementation of a scalable community discussion platform.

Key Achievements

- Secure JWT Authentication
- Community Creation and Management
- Post Creation and Feed System
- Cloudinary Image Upload Integration
- Voting and Engagement Mechanism
- Comment and Discussion Features
- PostgreSQL Relational Database Design
- Full-Stack Frontend Integration
- Layered Software Architecture

The project draws inspiration from platforms like Reddit and Stack Overflow while introducing a custom implementation focused on community-driven discussions, media-supported content creation, and secure user interaction.

This makes the platform practical, scalable, and industry-relevant while showcasing strong backend development, database design, API development, cloud integration, authentication, and full-stack engineering skills.